

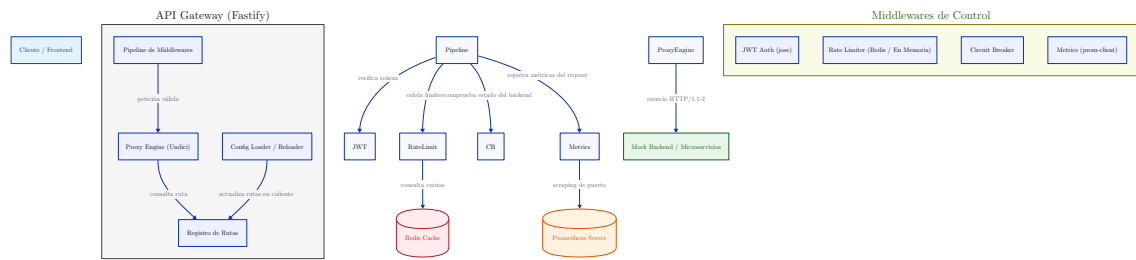
Este documento detalla la arquitectura de software, el stack tecnológico y la organización interna del API Gateway HTTP Modular. Está diseñado para ofrecer una comprensión profunda a ingenieros de software e ingenieros de infraestructura.

El Gateway está construido bajo la plataforma Node.js utilizando TypeScript y un conjunto de librerías de alto rendimiento y robustez comprobada en producción.

Componente	Tecnología	Versión
Propósito / Rol		
Entorno de Ejecución	Node.js	v20 LTS
Plataforma de ejecución de JavaScript del lado del servidor.	Lenguaje	TypeScript
v6.0.3	Lenguaje tipado que compila a JavaScript moderno.	Framework Web
Fastify	v5.8.5	Servidor HTTP de bajísima sobrecarga y alta velocidad de procesamiento.
Cliente de Proxy	Undici	v6.26.0
Cliente HTTP/1.1 y HTTP/2 oficial de Node.js, optimizado para alto rendimiento y pool de sockets.	Base de Datos / Caché	Redis (ioredis)
v5.10.1	Almacenamiento clave-valor en memoria utilizado para la limitación de tasa (Rate Limiting) distribuido.	Autenticación
jose	v6.2.3	Implementación ligera y sin dependencias nativas para firmas y validaciones JWT (JSON Web Tokens).
Validación de Esquemas	Zod	v4.4.3
Declaración y validación estricta de esquemas de configuración y peticiones.	Monitoreo / Métricas	prom-client
v15.1.3	Exportador de métricas compatible con el formato scrape de Prometheus.	Logs

Componente	Tecnología	Versión
Propósito / Rol		
Pino	v10.3.1	Motor de registro JSON extremadamente rápido y de bajo impacto en CPU.
Motor de Pruebas	Vitest	v4.1.7
Framework de testing rápido y nativo con soporte para ESM y TypeScript.		

El siguiente diagrama ilustra el flujo de una solicitud desde el cliente a través del API Gateway, detallando cómo se ejecutan los middlewares de control antes de que el motor de proxy reenvíe la petición al microservicio correspondiente.



La distribución física del código sigue un enfoque modular centrado en responsabilidades desacopladas, lo que permite añadir middlewares o modificar el motor de proxy de manera aislada.

```
Gateway/
├── .agent/                \# Configuración y skills del asistente de desarrollo
├── config/                \# Archivos de configuración del Gateway (ej.
gateway.yaml)
├── docker/                \# Recursos de Dockerización y Orquestación
│   ├── grafana/           \# Dashboards y configuración de Grafana para métricas
│   └── prometheus/        \# Configuración de scrape y jobs de Prometheus
├── src/                   \# Código fuente en TypeScript
│   ├── config/            \# Carga, validación con Zod y monitorización de recargas
│   ├── errors/            \# Definiciones y manejador global de errores del Gateway
│   ├── logger/           \# Módulo de logging centralizado con Pino
│   ├── middleware/        \# Canalización (pipeline) y middlewares individuales
│   └── proxy/             \# Motor de reenvío inverso de red (conexiones
│                           persistentes con Undici)
```

```
|   ├── routing/           \# Tabla de rutas y lógica de emparejamiento (matching)
|   ├── index.ts          \# Punto de entrada de la aplicación
|   └── server.ts         \# Configuración inicial del servidor de Fastify
└── tests/                \# Suite de pruebas unitarias y de integración
    ├── package.json      \# Manifiesto de dependencias y scripts npm/pnpm
    └── pnpm-lock.yaml     \# Archivo de bloqueo de versiones del gestor pnpm
```

Requisitos Previos

- Node.js v20 o superior.
- **pnpm** instalado globalmente (`npm install -g pnpm` o vía corepack).
- Docker y Docker Compose (opcional, para entornos de producción y pruebas).

Instalación de dependencias

```
pnpm install
```

Ejecutar en modo desarrollo

```
pnpm dev
```

Compilar para producción

```
pnpm build
```

Ejecutar la aplicación compilada

```
pnpm start
```

Ejecutar pruebas

```
\# Ejecutar todas las pruebas con Vitest
pnpm test

\# Ejecutar pruebas con reporte de cobertura
pnpm test:coverage
```